

FinShare — Security Enhancements

Phase 2 Backend Requirements / For: Abhilash Thomas / April 3, 2026

Hi Abhilash, once the Phase 1 auth work from the Master doc is complete, these four items harden the security and reliability of FinShare further. They are listed in recommended build order.

Item	Priority	Section
1. Rate Limiting + Account Lockout	Phase 2 — do after Phase 1 is live	Section 1
2. Email Token Expiry Handling	Phase 2 — pairs with email verification	Section 2
3. Audit Logs	Phase 2 — important for monitoring	Section 3
4. JWT Refresh Tokens	Phase 3 — plan now, build when ready	Section 4

Security Baseline — Applies to the Entire API

1. HTTPS / TLS:
All APIs must enforce HTTPS (TLS 1.2+) and reject plain HTTP requests.
In Program.cs, ensure `app.UseHttpsRedirection()` is present.
Azure App Service enforces HTTPS by default — confirm it is enabled in the portal.
2. JWT Signing Key:
The JWT signing key must be stored securely in environment variables or
Azure Key Vault — never hardcoded in `appsettings.json` or source code.
Use: `builder.Configuration["Jwt:SecretKey"]`
Store the actual value in Azure App Service > Configuration > Application settings.

Section 1 — Rate Limiting & Account Lockout

Rate limiting prevents brute-force attacks — someone repeatedly trying passwords, or spamming the register endpoint. Account lockout is a secondary line of defence that temporarily locks an account after too many failed login attempts.

1a. Built-in .NET 8 Rate Limiting (Program.cs)

ASP.NET Core .NET 8 includes built-in rate limiting. No extra NuGet package needed. Add this to Program.cs:

```
using Microsoft.AspNetCore.RateLimiting;  
using System.Threading.RateLimiting;
```

```
// Add before builder.Build():
builder.Services.AddRateLimiter(options =>
{
    // Login: max 5 requests per IP per minute
    options.AddFixedWindowLimiter("login", opt => {
        opt.Window          = TimeSpan.FromMinutes(1);
        opt.PermitLimit      = 5;
        opt.QueueLimit       = 0;
        opt.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
    });

    // Register: max 3 requests per IP per hour
    options.AddFixedWindowLimiter("register", opt => {
        opt.Window          = TimeSpan.FromHours(1);
        opt.PermitLimit      = 3;
        opt.QueueLimit       = 0;
    });

    // Forgot password: max 3 requests per IP per 15 minutes
    options.AddFixedWindowLimiter("forgotpw", opt => {
        opt.Window          = TimeSpan.FromMinutes(15);
        opt.PermitLimit      = 3;
        opt.QueueLimit       = 0;
    });

    // Return 429 Too Many Requests when limit is hit
    options.OnRejected = async (context, token) => {
        context.HttpContext.Response.StatusCode = 429;
        await context.HttpContext.Response.WriteAsJsonAsync(
            new { success = false, message = "Too many attempts. Please wait
and try again." },
            token);
    };
});

// Add after builder.Build():
app.UseHttpsRedirection(); // Enforce HTTPS – reject plain HTTP
app.UseForwardedHeaders(...); // Real IP behind proxy – see Section 3e
app.UseRateLimiter();
```

Note — Rate Limiter Storage & Multi-Instance Deployment

The built-in .NET rate limiter stores counters IN MEMORY. This works perfectly for a single server, which FinShare currently runs on. If the app is later scaled to multiple instances (load balancer / multiple pods), in-memory counters will NOT be shared – each server counts independently, which weakens the limits significantly. Future fix (when needed): Switch to distributed rate limiting using Redis. NuGet: StackExchange.Redis + a Redis-backed rate limit policy This is NOT needed now – note it as a Phase 3 scaling task.

1b. Apply Rate Limiters to Auth Endpoints

```
// In AuthController.cs - add [EnableRateLimiting] attributes:

[HttpPost("login")]
[EnableRateLimiting("login")]
public async Task<IActionResult> Login([FromBody] LoginRequest request) { ... }

[HttpPost("register")]
[EnableRateLimiting("register")]
public async Task<IActionResult> Register([FromBody] RegisterRequest request) {
    ... }

[HttpPost("forgot-password")]
[EnableRateLimiting("forgotpw")]
public async Task<IActionResult> ForgotPassword([FromBody]
ForgotPasswordRequest request) { ... }
```

Phase 2.1 Improvement — Add User+IP Combined Limiter for Login

IP-only rate limiting has two edge cases:

- Shared office / public Wi-Fi: all users share one IP → all get blocked together
- VPN: attacker switches IPs to bypass the limit

Better approach for login: combine IP + username in the key:

```
options.AddFixedWindowLimiter("login-user", opt => {
    opt.Window      = TimeSpan.FromMinutes(5);
    opt.PermitLimit = 10;
});
// In the Login action, before processing:
var key = $"{GetClientIP()}:{request.UserName}";
// Apply limiter using that key (requires custom policy - see Fido2
docs)
This way a shared IP only blocks that specific username, not all
users.
Implement this as a Phase 2.1 follow-up after the basic rate limiter
is live.
```

1c. Account Lockout — Database Changes

In addition to IP rate limiting, lock a specific account after 5 failed logins. This protects against attacks from multiple IPs targeting one account.

```
ALTER TABLE Users
ADD FailedLoginCount INT NOT NULL DEFAULT 0,
LockoutUntil DATETIME NULL;
```

1d. Stored Procedure — sp_RecordFailedLogin

```
CREATE OR ALTER PROCEDURE sp_RecordFailedLogin
@UserName NVARCHAR(200)
```

```

AS BEGIN
    UPDATE Users
        SET FailedLoginCount = FailedLoginCount + 1,
            -- Lock for 15 minutes after 5 failed attempts
            LockoutUntil = CASE
                WHEN FailedLoginCount + 1 >= 5
                THEN DATEADD(MINUTE, 15, GETUTCDATE())
                ELSE LockoutUntil
            END
        WHERE UserName = @UserName;
END

```

1e. Stored Procedure — sp_ClearFailedLogins

```

CREATE OR ALTER PROCEDURE sp_ClearFailedLogins
    @UserName NVARCHAR(200)
AS BEGIN
    UPDATE Users
        SET FailedLoginCount = 0,
            LockoutUntil = NULL
        WHERE UserName = @UserName;
END

```

1f. Update sp_ValidateUser (or Login Controller)

Before validating the password, check if the account is locked. After a successful login, clear the failed count.

```

-- Add to login logic (in sp_ValidateUser or in controller):

-- 1. Check lockout FIRST:
IF EXISTS (
    SELECT 1 FROM Users
    WHERE UserName = @UserName
    AND LockoutUntil > GETUTCDATE()
)
BEGIN
    -- Return: 'Account is temporarily locked. Please try again later.'
    RETURN;
END

-- 2. If password wrong: call sp_RecordFailedLogin
-- 3. If password correct: call sp_ClearFailedLogins, then issue JWT

```

Password Security Reminder

Confirm that all passwords are hashed using BCrypt with a minimum of 10 rounds.

This applies to: registration, password reset, and any future password change flow.

```

// In C# - BCrypt hashing (BCrypt.Net-Next NuGet package):
string hash = BCrypt.Net.BCrypt.HashPassword(password, workFactor:
10);

```

```
// Verify:
bool valid = BCrypt.Net.BCrypt.Verify(inputPassword, storedHash);
BCrypt is already used in registration – just confirm reset-password
does the same.
Never store plain text or MD5/SHA1 hashed passwords.
```

What Manus needs to do for rate limiting

```
Show a friendly message when 429 Too Many Requests is received:
if (response.status === 429) {
    toast.error('Too many attempts. Please wait a moment and try
again.');
```

For account lockout – the backend returns 401 with the lockout message.
Manus just needs to display whatever message the backend sends.

Section 2 — Email Token Expiry Handling

When a verification or reset link expires (24h for email verify, 1h for password reset), the user needs to request a new one. There are two parts: a resend endpoint on the backend, and periodic cleanup of stale tokens.

2a. Endpoint — POST /api/Auth/resend-verification

Endpoint: POST /api/Auth/resend-verification [No auth required]

```
Request : { email: string }
Response (always) :
  { success: true, message: 'If your email is registered, a new
  verification link has been sent.' }
Logic:
  1. Look up user by email.
  2. If IsEmailVerified = 1 already → still return success (do
  nothing).
  3. If not verified: generate new token, update via
  sp_SaveEmailVerifyToken,
  send verification email.
  4. Always return success (do not expose whether email exists).
Rate limit: apply the 'forgotpw' limiter (3 per 15 min per IP).
```

2b. Periodic Token Cleanup — SQL Agent Job or Background Service

Expired tokens sitting in the DB are harmless but messy. Clean them up periodically. Two options:

Option A — SQL Agent Job (simplest, no code changes):

```
-- Run this daily via SQL Server Agent:
UPDATE Users
  SET EmailVerifyToken = NULL,
      EmailVerifyExpiry = NULL
WHERE EmailVerifyExpiry < GETUTCDATE()
```

```

AND IsEmailVerified      = 0;

UPDATE Users
    SET PasswordResetToken = NULL,
        PasswordResetExpiry = NULL
WHERE PasswordResetExpiry < GETUTCDATE();

```

Option B — .NET Background Service (if no SQL Agent access):

```

// Add a hosted background service that runs every hour:
public class TokenCleanupService : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await Task.Delay(TimeSpan.FromHours(1), stoppingToken);
            // Call a cleanup SP here
            await _db.ExecuteAsync("sp_CleanupExpiredTokens");
        }
    }
}

// Register in Program.cs:
builder.Services.AddHostedService<TokenCleanupService>();

```

```

-- sp_CleanupExpiredTokens:
CREATE OR ALTER PROCEDURE sp_CleanupExpiredTokens AS BEGIN
    UPDATE Users SET EmailVerifyToken = NULL, EmailVerifyExpiry = NULL
    WHERE EmailVerifyExpiry < GETUTCDATE() AND IsEmailVerified = 0;

    UPDATE Users SET PasswordResetToken = NULL, PasswordResetExpiry = NULL
    WHERE PasswordResetExpiry < GETUTCDATE();
END

```

What Manus needs to do for email expiry

1. VerifyEmailConfirm screen already handles expired tokens (shows error + back to login).
2. VerifyEmailPending screen already has a Resend button.
3. For reset password: if token expired, show message:
'This link has expired. Please request a new reset link.'
With a button: 'Back to Forgot Password'

Section 3 — Audit Logs

Audit logs record every important security event — who logged in, from where, what failed. This is essential for debugging issues (e.g., 'why can't this user log in?') and spotting suspicious activity.

3a. Create AuditLogs Table

```
CREATE TABLE AuditLogs (  
    Id          BIGINT          IDENTITY PRIMARY KEY,  
    MemberId    INT             NULL,          -- NULL if user not found (failed  
login attempt)  
    Action      NVARCHAR(50)    NOT NULL,      -- see action types below  
    IPAddress   NVARCHAR(50)    NULL,  
    UserAgent   NVARCHAR(500)   NULL,  
    Success     BIT             NOT NULL,  
    Details     NVARCHAR(500)   NULL,          -- optional extra info  
    CreatedAt   DATETIME        NOT NULL DEFAULT GETUTCDATE()  
);  
  
-- Index for quick lookup by member:  
CREATE INDEX IX_AuditLogs_MemberId ON AuditLogs (MemberId);  
CREATE INDEX IX_AuditLogs_CreatedAt ON AuditLogs (CreatedAt);
```

3b. Action Types to Log

Action	Description
LOGIN_SUCCESS	User logged in successfully
LOGIN_FAILED	Wrong password or username not found
LOGIN_BLOCKED	Account locked due to too many failed attempts
REGISTER	New user registered
EMAIL_VERIFIED	User verified their email address
EMAIL_VERIFY_FAILED	Email verification failed (expired or invalid token)
FORGOT_PASSWORD	User requested a password reset link
PASSWORD_RESET	User successfully reset their password
PASSWORD_RESET_FAILED	Reset attempt failed (expired or invalid token)
LOGOUT	User logged out (refresh token revoked — Phase 3)

3c. Stored Procedure — sp_WriteAuditLog

```
CREATE OR ALTER PROCEDURE sp_WriteAuditLog  
    @MemberId INT = NULL,  
    @Action NVARCHAR(50),  
    @IPAddress NVARCHAR(50) = NULL,  
    @UserAgent NVARCHAR(500) = NULL,  
    @Success BIT,  
    @Details NVARCHAR(500) = NULL  
AS BEGIN  
    INSERT INTO AuditLogs  
        (MemberId, Action, IPAddress, UserAgent, Success, Details)  
    VALUES  
        (@MemberId, @Action, @IPAddress, @UserAgent, @Success, @Details);  
END
```

3d. Audit Log Retention Policy

Audit logs grow fast — a busy app can generate thousands of rows per day. Without a retention policy the table will become a performance problem. Recommended approach:

Data	Retention	Reason
Keep in primary table	90 days	Fast queries, small footprint for recent data
Archive to AuditLogs_Archive	1 year	Cheap long-term storage, rarely queried
Delete archived data	After 1 year	Balance legal/compliance needs with storage cost

```
-- Add to the sp_CleanupExpiredTokens job (or a separate daily job):

-- Step 1: Archive rows older than 90 days
INSERT INTO AuditLogs_Archive
SELECT * FROM AuditLogs WHERE CreatedAt < DATEADD(DAY, -90, GETUTCDATE());

-- Step 2: Delete archived rows from primary table
DELETE FROM AuditLogs WHERE CreatedAt < DATEADD(DAY, -90, GETUTCDATE());

-- Step 3: Delete from archive after 1 year
DELETE FROM AuditLogs_Archive WHERE CreatedAt < DATEADD(YEAR, -1,
GETUTCDATE());
```

Create AuditLogs_Archive with the same schema as AuditLogs. This keeps the primary table lean for fast lookups while preserving history.

3e. How to Get IP Address and User Agent in Controller

⚠ Important — IP Address Behind Azure / Nginx / Load Balancer

HttpContext.Connection.RemoteIpAddress gives the WRONG IP when deployed behind a reverse proxy (Azure App Service, Nginx, load balancer). It returns the proxy internal IP, not the real user IP.
Fix — add this to Program.cs BEFORE app.UseRateLimiter():

```
using Microsoft.AspNetCore.HttpOverrides;
app.UseForwardedHeaders(new ForwardedHeadersOptions {
    ForwardedHeaders = ForwardedHeaders.XForwardedFor
    | ForwardedHeaders.XForwardedProto
});
```

```
// In AuthController.cs — CORRECT helper method:
private string GetClientIP()
{
    // Read real client IP when behind Azure / Nginx / load balancer
    var forwarded = Request.Headers["X-Forwarded-For"].FirstOrDefault();
    if (!string.IsNullOrEmpty(forwarded))
        return forwarded.Split(',')[0].Trim();
    // Fallback for direct connections (local dev / no proxy)
    return HttpContext.Connection.RemoteIpAddress?.ToString() ?? "unknown";
}
```



```
// ❌ DO NOT use this alone — wrong behind a proxy:
// HttpContext.Connection.RemoteIpAddress?.ToString()

private string GetUserAgent() =>
    Request.Headers["User-Agent"].ToString();

// Usage example in Login action:
await _db.ExecuteAsync("sp_WriteAuditLog", new {
    MemberId = user?.MemberId,
    Action   = loginSuccess ? "LOGIN_SUCCESS" : "LOGIN_FAILED",
    IPAddress = GetClientIP(),
    UserAgent = GetUserAgent(),
    Success   = loginSuccess,
    Details   = loginSuccess ? null : "Invalid password"
});
```

Note — Audit Logs are backend-only

Manus does not need to do anything for audit logs. They are written automatically by the backend on every auth event. No new API endpoints are needed at this stage. In the future, an admin panel could expose:
 GET /api/Admin/audit-logs?memberId=X&from=...&to=...
 But this is not needed now.

Section 4 — JWT Refresh Tokens [Phase 3 — Plan Now, Build Later]

Currently, the JWT access token is long-lived (however long it is set to expire). Refresh tokens allow us to issue short-lived JWTs (e.g., 15 minutes to 1 hour) and silently renew them behind the scenes — keeping users logged in without them noticing, and improving security by limiting how long a stolen token is valid.

4a. How It Works

Token Type	Description
Access Token	Short-lived JWT (15–60 minutes). Used in every API request as Authorization: Bearer header.
Refresh Token	Long-lived opaque random string (7–30 days). Stored securely. Used ONLY to request a new access token.

Refresh Token Flow

1. Login → backend returns { accessToken, refreshToken }
2. Frontend stores both (accessToken in memory, refreshToken in httpOnly cookie or secureStorage)
3. On every API call: send accessToken in Authorization header
4. When API returns 401 (expired):

- Frontend calls POST /api/Auth/refresh with { refreshToken }
- Backend validates refresh token, issues new accessToken (+ rotates refreshToken)
- Frontend retries the original request with new accessToken

5. Logout: call POST /api/Auth/revoke to invalidate the refresh token in DB

- Server-side logout – stolen tokens can be revoked

4b. Database Changes

```
CREATE TABLE UserRefreshTokens (
    Id INT IDENTITY PRIMARY KEY,
    MemberId INT NOT NULL REFERENCES Users(MemberId),
    Token NVARCHAR(200) NOT NULL UNIQUE,
    ExpiresAt DATETIME NOT NULL,
    IsRevoked BIT NOT NULL DEFAULT 0,
    CreatedAt DATETIME NOT NULL DEFAULT GETUTCDATE(),
    DeviceInfo NVARCHAR(200) NULL -- optional: track device/browser
);

CREATE INDEX IX_RefreshTokens_Token ON UserRefreshTokens(Token);
CREATE INDEX IX_RefreshTokens_MemberId ON UserRefreshTokens(MemberId);
```

4c. New / Updated Endpoints

Endpoint	Change
POST /api/Auth/login	UPDATE — also returns refreshToken alongside JWT accessToken
POST /api/Auth/refresh	NEW — takes { refreshToken }, returns new { accessToken, refreshToken }
POST /api/Auth/revoke	NEW — takes { refreshToken }, marks it as revoked in DB (server-side logout)

4d. Stored Procedures

```
-- sp_SaveRefreshToken: called on login
CREATE OR ALTER PROCEDURE sp_SaveRefreshToken
    @MemberId INT,
    @Token NVARCHAR(200),
    @ExpiresAt DATETIME,
    @DeviceInfo NVARCHAR(200) = NULL
AS BEGIN
    INSERT INTO UserRefreshTokens (MemberId, Token, ExpiresAt, DeviceInfo)
    VALUES (@MemberId, @Token, @ExpiresAt, @DeviceInfo);
END

-- sp_ValidateRefreshToken: called on /refresh
CREATE OR ALTER PROCEDURE sp_ValidateRefreshToken
    @Token NVARCHAR(200)
AS BEGIN
    SELECT MemberId, ExpiresAt, IsRevoked
    FROM UserRefreshTokens
```

```

        WHERE Token = @Token;
END

-- sp_RevokeRefreshToken: called on /revoke (logout)
CREATE OR ALTER PROCEDURE sp_RevokeRefreshToken
    @Token NVARCHAR(200)
AS BEGIN
    UPDATE UserRefreshTokens SET IsRevoked = 1 WHERE Token = @Token;
END

-- sp_RevokeAllUserTokens: called if user changes password or account is
compromised
CREATE OR ALTER PROCEDURE sp_RevokeAllUserTokens
    @MemberId INT
AS BEGIN
    UPDATE UserRefreshTokens SET IsRevoked = 1 WHERE MemberId = @MemberId;
END

```

4e. Token Rotation

Each time a refresh token is used to get a new access token, issue a new refresh token and revoke the old one (this is called refresh token rotation). If someone steals and uses an old refresh token, the system detects it because the original user's rotation will have already invalidated it.

What Manus needs to do for Refresh Tokens

1. Store accessToken in memory (React state / variable) – not localStorage.
Memory storage is safer against XSS attacks.
2. Store refreshToken – PREFERRED option:
 - ✅ httpOnly cookie (set by backend with Set-Cookie header)
 - JavaScript cannot read it → immune to XSS token theft
 - Backend sets: Set-Cookie: refreshToken=XXX; HttpOnly; Secure; SameSite=Strict
 - Acceptable fallback for PWA only:
 - ⚠️ localStorage – simpler but readable by JavaScript
 - Use only if httpOnly cookie causes PWA/CORS issues
 - Add note in code: // TODO: migrate to httpOnly cookie
3. Add an Axios/fetch interceptor:
 - On 401: call POST /api/Auth/refresh
 - On success: retry original request with new accessToken
 - On refresh failure: clear tokens, redirect to /login
4. Update logout to call POST /api/Auth/revoke before clearing local tokens.

4f. Recommended Access Token Expiry

Setting	Value	Reason
Access Token expiry	15-60 minutes	Short — limits damage if stolen
Refresh Token expiry	7-30 days	Comfortable for users, revocable
Remember Me option	30 days refresh	Optional — longer refresh token if user ticks 'Keep me signed in'

Section 5 — Future Enhancements [Strategic Roadmap]

These are not immediate work items — they are planned for a later phase. Documenting them now ensures the architecture is designed with room to grow.

Feature	Phase	Description
Device Tracking & Session Management	Phase 4	Track which devices a user is logged in from (device name, last active, location). Allow users to view and revoke individual sessions from a 'My Devices' screen. Builds on UserRefreshTokens table (Section 4) — add DeviceName + LastUsedAt columns.
Suspicious Login Alerts	Phase 4	Detect unusual login patterns: new country, new device, login at unusual hours. Send an email alert: 'New sign-in to your FinShare account from iPhone in Singapore. If this wasn't you, secure your account here.' Requires storing last-known IP + geolocation lookup (e.g., ip-api.com free tier).
Admin Audit Dashboard	Phase 4	A simple admin screen showing recent auth events filterable by user, date, and action type. Endpoint: GET /api/Admin/audit-logs?memberId=&from=&to=&action= Requires admin role in JWT claims. Manus builds the read-only dashboard UI.

Summary — All Tasks

#	Task	Priority	Ref
1	Program.cs — UseForwardedHeaders() + AddRateLimiter()	Phase 2	§1a
2	AuthController — [EnableRateLimiting] on login/register/forgotpw	Phase 2	§1b
3	Login: Phase 2.1 — user+IP combined limiter	Phase 2.1	§1b
4	ALTER TABLE Users — FailedLoginCount + LockoutUntil	Phase 2	§1c
5	sp_RecordFailedLogin + sp_ClearFailedLogins	Phase 2	§1d/e
6	Login logic — check LockoutUntil, record/clear failed attempts	Phase 2	§1f
7	Confirm BCrypt with minimum 10 rounds in all password endpoints	Phase 2	§1
8	POST /api/Auth/resend-verification endpoint	Phase 2	§2a
9	sp_CleanupExpiredTokens + SQL Agent Job or BackgroundService	Phase 2	§2b
10	CREATE TABLE AuditLogs + CREATE TABLE AuditLogs_Archive	Phase 2	§3a
11	sp_WriteAuditLog	Phase 2	§3c
12	Add audit log writes to all auth controller actions	Phase 2	§3e
13	Add daily archive+cleanup job for 90-day retention policy	Phase 2	§3d

14	CREATE TABLE UserRefreshTokens	Phase 3	§4b
15	sp_SaveRefreshToken + sp_ValidateRefreshToken + sp_RevokeRefreshToken	Phase 3	§4d
16	Update POST /api/Auth/login — return refreshToken alongside JWT	Phase 3	§4c
17	POST /api/Auth/refresh endpoint (with token rotation)	Phase 3	§4c
18	POST /api/Auth/revoke endpoint + set httpOnly cookie on login	Phase 3	§4e
19	Device tracking — add DeviceName + LastUsedAt to RefreshTokens	Phase 4	§5
20	Suspicious login detection + email alert	Phase 4	§5
21	GET /api/Admin/audit-logs — admin dashboard endpoint	Phase 4	§5

— Joseph Xavier / FinShare Project / April 3, 2026